RDF graph → SPARQL query Q3 (abridged)

```
SELECT
  (COUNT(DISTINCT ?record) AS ?biallelicSnvCount)
  (SUM(IF(isTransition(?ref, ?alt), 1, 0))
                          AS ?transitionCount)
  (SUM(IF(isTransition(?ref, ?alt), 0, 1))
                          AS ?transversionCount)
WHERE {
  ?record a vcfc:VCFRecord ;
          vcfc:ref ?refLiteral ;
          vcfc:alt ?altLiteral .
  BIND(UCASE(STR(?refLiteral)) AS ?ref)
  BIND(UCASE(STR(?altLiteral)) AS ?alt)
  FILTER(REGEX(?ref, "^[ACGT]$") &&
         REGEX(?alt, "^[ACGT]$") && ?ref != ?alt)
}
```

`isTransition(r, a)` abbreviates the query's test that (r, a) is A→G, G→A, C→T, or T→C.

Source VCF → cyvcf2 oracle (pseudo-code)

```
TRANSITIONS = {(A,G), (G,A), (C,T), (T,C)}
snvs = transitions = transversions = 0

for record in cyvcf2.VCF(source_vcf):
    ref = upper(record.REF)
    alt = upper(join(record.ALT, ","))
    if ref in {A,C,G,T} and alt in {A,C,G,T}
            and ref != alt:
        snvs += 1
        if (ref, alt) in TRANSITIONS:
            transitions += 1
        else:
            transversions += 1

expected = {biallelicSnvCount: snvs,
            transitionCount: transitions,
            transversionCount: transversions}
```

SPARQL result

`biallelicSnvCount, transitionCount, transversionCount`

Expected result

`biallelicSnvCount, transitionCount, transversionCount`

Normalize and compare: the three counts must agree exactly (PASS, otherwise MISMATCH); Ti/Tv = transitions / transversions is derived only after the counts agree.